

Java GUI Programming Formative Assessment Marking Guide

Administration, rubric, feedback, and pre-test post-test alignment

Assessment	Java GUI Programming using MagikLayout
Target group	First- or second-year students learning Java Swing layout managers
Recommended duration	Pre-test 10 minutes; activity 60 minutes; post-test 10 minutes
Student evidence	Completed hard-copy booklet, lecturer checkpoints, and pre-test/post-test submissions

Recommended decision. Use the booklet as a low-stakes formative assessment and use the online pre-test and post-test as the learning measure. The assessment deliberately records predictions before observation, requires students to manipulate the layout, and asks them to explain the generated Java. This creates evidence of reasoning rather than attendance alone.

Learning outcome alignment

Outcome	Student evidence	Task	Marks
Predict placement and resizing	Prediction and observation records for BorderLayout	1	8
Compare layout-manager behaviour	FlowLayout and GridLayout comparison	2	8
Build and explain a nested layout	Conflict repair, component structure, generated Java	3	10
Respond to deterministic feedback	Reflow and Reverse challenge records	4	8
Transfer knowledge to a new design	Component tree or wireframe and justification	5	6

Administration protocol

1. Prepare one numbered student booklet for each student. Record the booklet number beside the student's name or registration number.
2. Release the pre-test first. Require one submission, hide correct answers, and close or remove access when the activity begins.

3. Give students the booklet and the MagikLayout URL. Read the assessment conditions aloud and start the 60-minute session.
4. Verify Task 1, Task 3, and Task 4 at the student's screen. Initial only after seeing the required state. The checkpoint is evidence of tool use, not an extra mark.
5. Collect the booklet before releasing the post-test. Keep students seated and ask them to complete the post-test once without reopening MagikLayout.
6. Export both Google Form response sheets and match pre-test/post-test records using registration number. Store the assessment marks separately from research-consent data.
7. Give corrective feedback at the next class. Re-teach concepts with weak item performance or common rubric errors.

Recommended controls for the master's study

- Use the same instructions, time limits, device conditions, and lecturer script for every class included in the study.
- Use a parallel post-test with the same constructs and difficulty as the pre-test, but change the component names and scenarios. Keep the scoring scheme identical.
- Do not show correct answers or item-level feedback between the pre-test and post-test.
- Record absences, incomplete attempts, technical failures, and whether a student used a hint. Decide exclusion rules before analysing results.
- Analyse paired scores only for students with a valid pre-test and post-test identifier. Report sample size, missing data, score change, and an appropriate paired statistical test with an effect size.
- Keep the post-use experience survey separate from course marks unless the institution's ethics approval and consent process explicitly permit a required research questionnaire.

Analytic marking rubric

Scoring principle. Award marks for accurate reasoning and evidence. A wrong initial prediction can still earn observation and explanation marks when the student clearly identifies the corrected rule.

Criterion	Full-credit evidence	Partial-credit guidance	Marks
Task 1 prediction	Three predictions recorded before testing and logically connected to layout behaviour.	1 mark per meaningful prediction; no credit for blank or copied-after-testing entries.	3
Task 1 observation and rule	Observations distinguish CENTER, horizontal edge regions, and vertical edge regions; explanation links size changes to BorderLayout.	Award 1 mark per accurate observation and up to 2 for the explanation/code evidence.	5
Task 2 observations	Comparison accurately distinguishes preferred-size wrapping from equal-size cells and records narrowing behaviour.	Award across the four comparison rows according to accuracy and completeness.	4
Task 2 selection	Chooses a suitable manager for compact wrapping controls and for an equal-cell keypad, with reasons.	2 marks for each justified choice; 1 for an unsupported correct choice.	4
Task 3 diagnosis	States that two direct occupants conflict in one BorderLayout region and records the visible result.	Award for accurate observation even if terminology is incomplete.	3
Task 3 repair	Nested JPanel is placed in SOUTH and contains both buttons under a suitable child layout.	Award for a structurally valid repair; deduct when both buttons are still direct frame children.	4
Task 3 explanation	Component hierarchy and generated Java show why the nested panel preserves both buttons.	Award for hierarchy, code evidence, and causal explanation.	3

Analytic marking rubric continued

Criterion	Full-credit evidence	Partial-credit guidance	Marks
Task 4 challenge evidence	Two requested modes attempted, first results and final results recorded, checkpoints verified.	Award separately for each completed challenge record.	4
Task 4 learning explanation	Identifies a specific misconception and explains how feedback changed the student's reasoning.	General praise of the tool without a Swing rule earns limited credit.	4
Task 5 structure	Wireframe or component tree uses valid nesting and labels layouts and placements.	Award for a workable partial structure even when one layout choice is weak.	3
Task 5 justification	Explains layout choices and gives a plausible resize prediction.	Award 1 mark per defensible explanation element.	3

Suggested performance bands

Score	Interpretation	Recommended feedback
34–40	Secure	Proceed to a more complex nested GUI task; ask the student to explain trade-offs.
26–33	Developing	Review one weak layout manager and repeat one challenge with explanation.
18–25	Needs support	Use a guided demonstration followed by prediction and retesting.
0–17	Foundational gaps	Re-teach container, child, layout manager, region, and resize concepts before reassessment.

Post-test blueprint

Build a parallel 10-item post-test. Use new interface scenarios while preserving the pre-test's constructs, number of options, scoring, and approximate reading demand. Randomise option order only when it does not change the meaning.

Items	Construct	Observable response	Cognitive demand
1–2	BorderLayout regions and resizing	Predict placement or identify the region that absorbs remaining space.	Apply
3–4	FlowLayout	Predict order, wrapping, preferred-size behaviour, or alignment.	Apply
5–6	GridLayout	Predict equal-cell sizing, rows, columns, order, or resize behaviour.	Apply
7–8	Nested panels and region conflict	Diagnose a missing component or choose a valid nested repair.	Analyse
9–10	Generated Java and layout selection	Interpret a short code fragment or choose a manager for a stated requirement.	Analyse

Data sheet structure

Field	Example or coding rule
participant_id	Use the same registration number or a study code in both tests.
class_batch	Pilot class, second batch, or another predefined group label.
pre_total and post_total	0–10; preserve item-level scores as separate columns.
assessment_total	0–40 from the hard-copy rubric.
hint_use	Record no/yes or the count if MagikLayout makes it available.
completion_status	Complete, incomplete, absent, or technical failure.

Feedback codes for faster marking

Code	Meaning	Feedback to student
BL1	Region behaviour unclear	Resize the frame again and compare CENTER with NORTH or SOUTH.
BL2	Two direct occupants in one region	Place a JPanel in the region, then add both controls to that panel.
FL1	Flow wrapping misunderstood	Record component preferred sizes, then narrow the container gradually.
GL1	Grid sizing misunderstood	Compare every cell after resizing; identify what remains equal.
NP1	Hierarchy unclear	Draw the frame, child panels, and controls as a tree before reading the Java.

Rationale and sources

Prediction before observation. Brown and Wilson recommend asking programming learners to make and record predictions before demonstrations. Tasks 1 and 2 therefore capture a prediction before the student resizes or switches layouts.

Immediate but interpretable feedback. Reviews of automated programming assessment distinguish formative feedback from grades alone and note the value of prompt feedback and repeated attempts. Tasks 3 and 4 require students to interpret MagikLayout's deterministic result and state the rule learned.

Authentic Swing behaviour. The Oracle Swing tutorials describe layout managers as controlling component size and position, identify FlowLayout wrapping, GridLayout equal-sized cells, and BorderLayout's region-based behaviour. The assessment uses these observable behaviours as its content domain.

References

- Brown, N. C. C., & Wilson, G. (2018). Ten quick tips for teaching programming. *PLOS Computational Biology*, 14(4), e1006023. <https://doi.org/10.1371/journal.pcbi.1006023>
- Keuning, H., Jeuring, J. T., & Heeren, B. (2018). A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education*, 19(1), Article 3. <https://doi.org/10.1145/3231711>
- Messer, M., Brown, N. C. C., Kölling, M., & Shi, M. (2024). Automated grading and feedback tools for programming education: A systematic review. *ACM Transactions on Computing Education*, 24(1), Article 10. <https://doi.org/10.1145/3636515>
- Oracle. (n.d.). Laying out components within a container. The Java Tutorials. <https://docs.oracle.com/javase/tutorial/uiswing/layout/>
- Van Petegem, C., Maertens, R., Strijbol, N., et al. (2023). Dodona: Learn to code with a virtual co-teacher that supports active learning. *SoftwareX*, 24, 101578. <https://doi.org/10.1016/j.softx.2023.101578>

Lecturer planning record

Class and date

Pre-test link or QR code

Post-test link or QR code

Experience survey and consent instructions

Changes made after the pilot and reason
